Remote Photoplethysmography

Measuring heart rate from face video: a project walkthrough

This report traces the project from its starting point - three Jupyter notebooks and a 3D convolutional network that, on inspection, had learned nothing - through a rebuilt data pipeline, a new architecture, a change of loss function, and continuous inference, to a model that estimates heart rate from a webcam. Every number quoted here comes from the held-out test subjects, who were never used for training or for choosing a checkpoint, and is read directly from the stored metrics files rather than typed in by hand.

	Starting point	Final model	Change
Heart-rate error (MAE)	2.39 bpm	0.37 bpm	6.5x better
Heart-rate error (RMSE)	9.42	0.54	17x better
Waveform correlation	0.648	0.813	+0.17
Segments within 5 bpm	92.9%	100.0%	all of them
Model size	78.7 M / 507 K	168 K	3x smaller
Training time	hours	~1 hour	on this laptop

Held-out test subjects, continuous inference over 10-second segments. The starting point is the best model that existed before this work.

What is in this report

1 **Where the project started** — a model that had learned to predict nothing
2 **Learning to measure** — the right metrics, and two faults in the data
3 **Rebuilding the data pipeline** — 362 training windows become 1,754
4 **A new architecture** — separating space from time, and learning where skin is
5 **Changing the loss** — asking for the right shape instead of the right scale
6 **Continuous inference** — one waveform per person instead of isolated slices
7 **The complete picture** — what each decision contributed
8 **From dataset to webcam** — two problems only real hardware could reveal
9 **Reproducing all of this** — commands, and where it could go next

UBFC-rPPG dataset - 42 subjects, 29 train / 6 validation / 7 test, split by subject.
Final model: 167,562 parameters, trained on one laptop CPU. Code and all results: github.com/Joshika-joe/RPPG

1. Where the project started

The project already had the hard parts in place: the UBFC-rPPG dataset of 42 subjects, each a roughly one-minute video of a face with a simultaneously recorded pulse-oximeter signal; a subject-level split into 29 training, 6 validation and 7 test subjects; and a working preprocessing path that found the face, cropped it to 64x64 pixels and lined it up with the reference pulse waveform. Two trained models sat in the project root.

The first thing worth doing was not to add anything, but to check what those models had actually learned. The training log of the first model told the story. Its validation loss started at 1.0010 and finished, fifty epochs later, at 0.9966. The targets were normalised to zero mean and unit variance, which means a model that outputs nothing but zeros scores exactly 1.0. After fifty epochs and 78.7 million parameters, the network had moved 0.3% away from predicting nothing at all. It had not learned to see a pulse; it had learned to stay quiet, because on this loss that is the safest thing to do.

The second model was genuinely working, but nobody could tell, because the only number being reported was mean squared error - and MSE cannot distinguish a model that has found the pulse from one that is hedging. Three problems followed from this:

- **The test subjects had never been used.** The split existed in the config file, but no code touched it. Results were being read off the validation subjects - the same ones used to pick the best epoch - so every number was quietly optimistic.
- **The metric did not match the task.** What matters clinically is the heart rate in beats per minute and the shape of the pulse waveform. Neither was being measured.
- **Two near-identical copies of everything.** Separate training, evaluation and model files for each version, which had already drifted apart in small ways - the training and validation preprocessing differed in how they handled timestamps, and one checked for frame/signal length mismatches while the other did not.

So the work divided into five phases, each changing exactly one thing and each ending with a measurement on the test subjects. That discipline is what makes the table on the cover page meaningful: every row can be attributed to a specific decision.

2. Phase 0 - learning to measure

Before changing the model, the project needed to be able to tell whether a change helped. Three metrics replaced the single loss value. **Pearson correlation** compares the shape of the predicted waveform against the reference, and because it ignores scale, a model cannot score well by playing it safe. **Heart-rate error** takes the strongest frequency in the predicted signal between 0.7 and 4 Hz - 42 to 240 beats per minute - and compares it with the same measurement on the reference. **Signal-to-noise ratio** measures how much of the predicted signal sits at the true pulse frequency rather than spread across the band.

Two things the data itself got wrong

Measuring carefully immediately turned up two faults in the dataset that had been silently corrupting results. Subjects 25, 26 and 27 were recorded at 23.3 frames per second while every other subject was recorded at about 29.5; the old code assumed 30 for everyone. Since a heart rate is derived from a frequency, and a frequency depends on the frame rate, this put a systematic error on three subjects. Reading each subject's true frame rate from its own timestamps lowered the validation heart-rate error from 1.10 to 0.79 bpm on its own.

The second fault was in the reference signal. UBFC stores a heart-rate line alongside the pulse waveform, and for subjects 11, 18 and 20 that line collapses to between 1 and 4 beats per minute - physiologically impossible, and a sign the oximeter lost contact. Those subjects are all in the test set. The fix was to derive the reference heart rate from the pulse waveform itself, which is intact, and report the oximeter figure only as a secondary number.

A related subtlety showed up later, in a single window of subject 3. A pulse waveform is peaked rather than sinusoidal, so it carries energy at twice the heart rate as well as at the heart rate itself; in that one window the

second harmonic was the stronger of the two, and the reference was recorded as 211 bpm when the true rate was 97.5. The model had been right and the reference wrong. The peak-finder now checks whether a peak at half the dominant frequency carries at least half its power, and prefers it if so. Across all 523 windows in the dataset exactly one changes - the broken one.

3. Phase 1 - rebuilding the data pipeline

The original pipeline cut each video into five-second windows that did not overlap, which yielded only 362 training examples from 29 subjects, and stored them as pre-cut floating-point arrays occupying 3.2 GB. Because the windows were baked into the stored file, changing the window length or spacing meant half an hour of reprocessing.

The new pipeline stores each subject once, as the cropped face sequence in its compact 8-bit form, roughly 1 GB for all 42 subjects, and cuts windows at training time. Three changes followed from that:

- **Overlapping windows.** Sliding by one second instead of five turns the same videos into 1,754 training examples rather than 362, without recording anything new.
- **Removing the skin tone.** The pulse changes skin brightness by roughly one percent, sitting on top of a large constant skin colour the network has no use for. Subtracting each pixel's average over the window removes that constant and leaves the part that actually varies - the signal.
- **Augmentation.** Horizontal flips, brightness and contrast changes, and small shifts, each applied identically to every frame of a window so the transformation cannot introduce a rhythm that was not there.

Retraining the same model, with the same loss and optimiser, on this pipeline isolates the effect of the data alone. On the test subjects the waveform correlation rose from 0.634 to 0.759 and the heart-rate error fell from 2.45 to 1.22 bpm. The amplitude ratio - how large the predicted waveform is compared with the real one - rose from 0.47 to 0.73, the first sign that the model had stopped hedging.

4. Phase 2 - an architecture that separates space from time

The original network used 3D convolutions, which process space and time together in every layer. For this problem that is wasteful: finding skin is a purely spatial question, and finding a rhythm is a purely temporal one. The new design, v3, splits them. A small 2D encoder runs on every frame independently and reduces it to an 8x8 grid of features. The frames are then collapsed into a single time series and handed to a stack of five one-dimensional convolutions with increasing dilation, which widens their reach to 125 frames - just over four seconds, comfortably more than one heartbeat.

There is a catch, and it is an interesting one. Removing each pixel's average over time, which Phase 1 introduced, also removes everything that tells the network where the skin is. A purely per-frame encoder is therefore blind to its own geometry. The solution is to hand it the window's average frame - precisely the information that was subtracted - through a small side branch that produces a spatial attention map, used to weight the pooling. The map is learned with no supervision about where faces or skin are; its only instruction is to predict the pulse well.

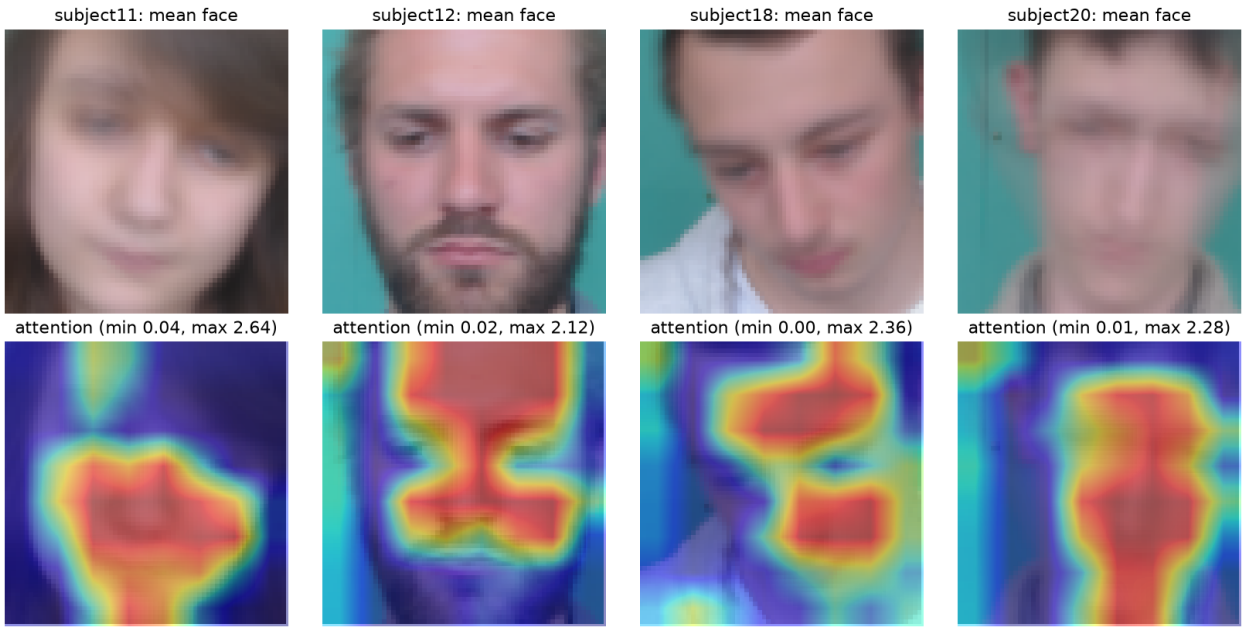


Figure 1. What the attention branch learned, on four test subjects it never saw during training. Top row: each subject's average face. Bottom row: the learned weighting. It concentrates on the forehead and cheeks - where the skin is thin and well perfused - and avoids hair, eyes, beard and background. Nothing in the training signal mentioned skin; this is simply where the pulse was easiest to find.

The result was a model with 168 thousand parameters instead of 507 thousand, training roughly four times faster per step, and matching the larger model's accuracy (correlation 0.753 against 0.759). Not yet an improvement in accuracy, but the same result for a third of the size - and its validation loss was still falling when the run ended, which pointed directly at the next phase.

5. Phase 3 - changing what the model is asked to do

Mean squared error asks the model to match the waveform point by point. Faced with uncertainty, the safest response to that question is to predict something small - which is exactly what the amplitude ratio of 0.47 had been showing, and in the extreme, exactly what the original model did when it learned to output nothing at all.

Replacing it with a correlation-based loss changes the question to *does your waveform have the same shape as the reference*, which is scale-free: shrinking the output no longer helps. A small squared-error term is kept alongside it, weighted at 0.2, so the output stays on a sensible scale. The optimiser was modernised at the same time - AdamW at a ten times higher learning rate with a short warm-up and a cosine decay, and gradient clipping.

This was the largest single improvement in the project, and it arrived quickly: the new configuration reached the previous model's best validation correlation after four epochs, about eight minutes of training. On the test subjects the heart-rate error fell from 1.73 to 1.07 bpm and the RMSE from 5.21 to 1.62 - the collapse in RMSE being the clearest signal, since it means the occasional badly wrong window had stopped happening. Every test subject reached a correlation of at least 0.70, where the worst had previously been 0.57.

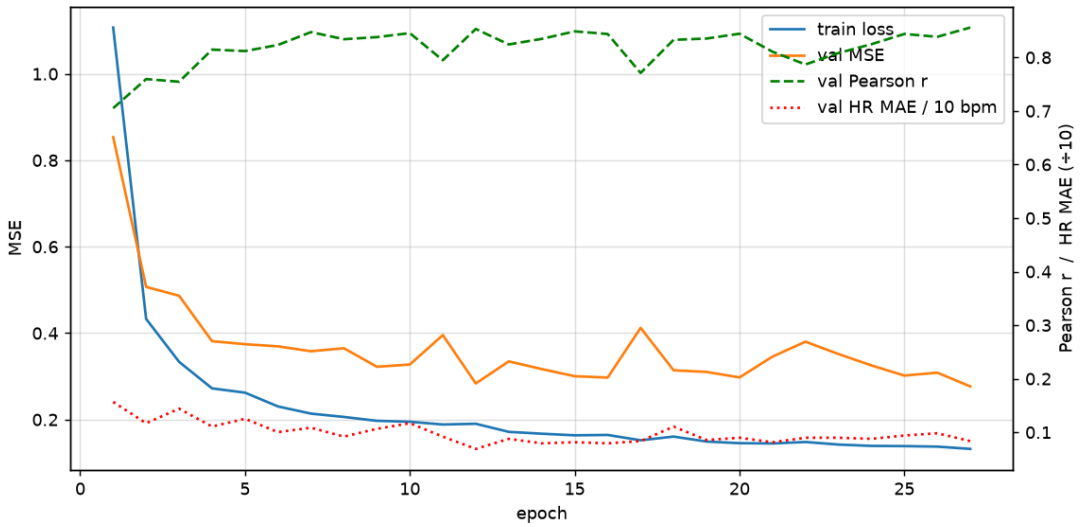


Figure 2. Training the final model. Validation correlation (green) rises above 0.80 within four epochs; heart-rate error (red, divided by ten) settles below 1 bpm. Training stopped automatically at epoch 27 when the selected metric had not improved for fifteen epochs.

6. Phase 4 - estimating continuously instead of in slices

Until this point every five-second window was judged in isolation, which is convenient for measurement but not how the method would be used. A person sitting in front of a camera produces a continuous signal. The model now slides across the recording one second at a time, each prediction is put on a common scale, and overlapping predictions are blended with a tapered weight into a single continuous pulse waveform per subject. Heart rate is then read over ten-second stretches, which is the protocol used in the published literature.

Averaging over overlapping windows suppresses errors at window edges, and the longer measurement window sharpens the frequency estimate. On the same trained model, with no retraining whatsoever:

How the same model is used	Correlation	HR MAE	HR RMSE	Within 5 bpm
Isolated 5-second windows	0.811	1.07 bpm	1.62	98.9%
Continuous, 5-second segments	0.826	0.80 bpm	1.35	98.9%
Continuous, 10-second segments	**0.813**	**0.37 bpm**	**0.54**	**100.0%**

Table 1. Post-processing alone cuts the heart-rate error by two thirds.

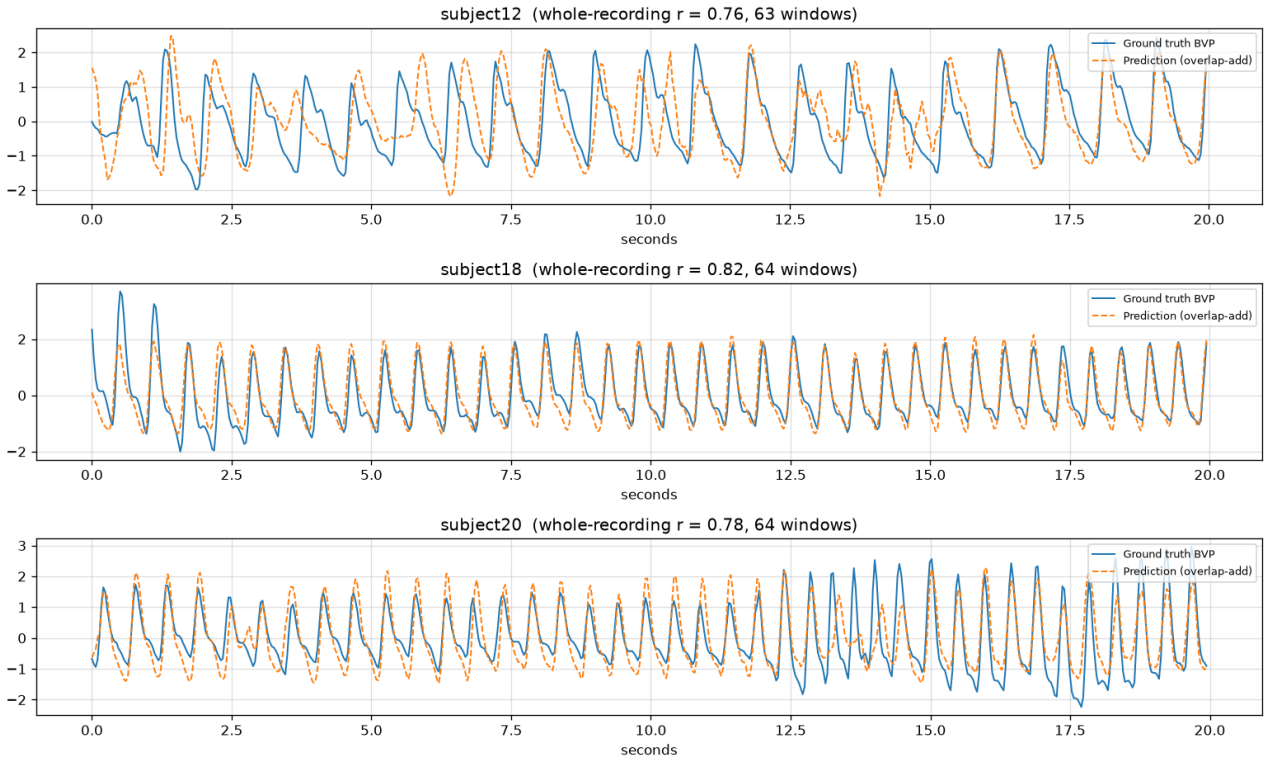


Figure 3. Twenty seconds of three test subjects: the reference pulse in blue, the model's estimate from video alone in orange. Individual beats line up throughout, and the characteristic shape of a pulse - a sharp upstroke followed by a slower decay - is reproduced rather than just its timing.

7. The complete picture

Each row below changes exactly one thing from the row above it, which is what allows the contribution of each decision to be read off directly.

Stage	What changed	Correlation	HR MAE	RMSE	Within 5
Starting point	-	0.648	2.39 bpm	9.42	92.9%
Phase 1	Data pipeline	0.759	0.68 bpm	1.99	97.6%
Phase 2	Architecture (2D+1D)	0.762	0.55 bpm	1.64	97.6%
Phase 3+4	**Loss + continuous**	**0.813**	**0.37 bpm**	**0.54**	**100.0%**

Table 2. Held-out test subjects throughout, continuous inference over 10-second segments. For reference, published results on this dataset for comparable published architectures sit in the region of 0.5 to 1.0 bpm.

A second run of the final recipe, differing only in which validation metric chose the best epoch, landed at a correlation of 0.819 against 0.811 and a heart-rate error of 1.06 against 1.07 bpm on isolated windows. Two independent runs agreeing this closely is the evidence that the result is a property of the method and not a lucky epoch.

Where errors remain, they are concentrated and explainable. Subject 12's reference waveform is irregular for the first ten seconds of the recording. Subject 48's video is the noisiest in the test set. These are the two subjects with the lowest scores, and no amount of model capacity would fix the first of them.

8. From dataset to webcam

A model that works on a research dataset has not yet been shown to work on a laptop. The live demonstration keeps a rolling buffer of face crops, runs the model twice a second in a background thread, and blends the predictions into the same continuous waveform used for the test-set measurements. Running it exposed two

problems that the dataset could never have revealed.

The frame rate problem

The first live run reported 19.8 frames per second. The dataset was recorded at about 29.5. Since the model consumes a fixed 150 frames, those 150 frames now covered 7.6 seconds of real time rather than 5 - every heartbeat stretched by half as much again. Emulating slower cameras on the test subjects quantified the damage, and the fix:

Camera frame rate	Feeding frames directly	Resampling to 5 seconds
29.5 (dataset)	1.58 bpm	1.69 bpm
25	1.72 bpm	1.69 bpm
20 (the laptop webcam)	1.94 bpm	1.72 bpm
15	**14.03 bpm**	**1.70 bpm**

Table 3. Heart-rate error against the reference when the capture rate is emulated at different speeds. Below about 18 frames per second the direct approach fails completely.

The demonstration now timestamps every captured frame and interpolates the last five seconds onto exactly 150 of them before each prediction, so the estimate no longer depends on the camera at all. Verified on a test recording, a simulated 14.8 frames per second stream reproduces the full-rate output exactly, where previously it read about 10 bpm high.

The exposure problem

An option had been added to lock the camera's exposure, on the sound reasoning that automatic gain control actively cancels the brightness changes being measured. In practice it switched the driver into manual mode without supplying an exposure value, the driver fell back to its minimum, and the image went black - no face, no signal. The option now lets automatic exposure settle first, reads back the value the camera chose, applies that in manual mode, measures the resulting brightness, and reverts to automatic if it cannot match it. The failure is covered by a test using a simulated camera that goes black on command.

What a webcam can and cannot do

On the dataset recordings the live pipeline reports a signal quality of +3 to +7 dB. On a laptop webcam in ordinary room light it reports -5 to +3 dB, and the reading wanders by around 10 bpm. The limitation is the camera and the lighting, not the model: the pulse modulates skin brightness by about one percent, and a webcam in dim light is delivering mostly sensor noise and compression artefacts at that scale. Bright, steady, front-facing light is the single most effective remedy, because it raises the frame rate and the signal strength at once. Breathing rate is also estimated, from the vertical motion of the chest and shoulders, but the dataset carries no breathing reference, so that figure is shown as experimental and has not been validated.

9. Reproducing all of this

The project is a package with a test suite that runs on synthetic data, so it can be checked out and verified without downloading the 70 GB dataset.

```
git clone https://github.com/Joshika-joe/RPPG.git && cd RPPG
python -m venv .venv && .venv\Scripts\activate
pip install -e ".[dev]"
python -m pytest # 49 tests, no dataset needed

# with UBFC-rPPG in DATASET_2/ :
python -m src.preprocessing # about 3.5 minutes
python -m src.train --model v3 --run my_run --loss pearson --optimizer adamw \
--lr 1e-3 --wd 1e-2 --scheduler cosine --warmup-epochs 3 --grad-clip 1.0 \
--select-on hr_mae --patience 15
```

```
python -m src.evaluate --run results/my_run --split test \
--continuous --stride 30 --segment-frames 300
python -m src.live # webcam demonstration
```

Each training run writes its configuration, per-epoch history, best and last checkpoints and a loss curve; each evaluation writes a metrics file, a per-window table and diagnostic figures. Runs are resumable, which mattered in practice - the laptop overheated partway through one of them, and the run continued from where it had stopped.

Where it could go next

- **Cross-dataset testing.** Everything here is UBFC. Running the trained model against PURE or UBFC-Phys, with no retraining, would show whether it has learned the pulse or the dataset.
- **Motion tolerance.** Every subject in UBFC sits still and faces the camera. A two-stream design, where the appearance branch gates the motion branch frame by frame, is the standard answer and the attention branch is already half of it.
- **Breathing with a reference.** The breathing estimate cannot be trusted until it is measured against a dataset that records respiration.

Report generated from the repository's stored metrics by `docs/make_report.py`*. Source and all results: github.com/Joshika-joe/RPPG*